

Имитационное моделирование

Лабораторная работа №2

АВТОР

Гашимов Кенан Мухтар оглы

ПРИНАДЛЕЖНОСТЬ

Российский университет дружбы народов

1 Сведения об авторе

- ФИО: Гашимов Кенан Мухтар оглы
- Группа: НКНбд-01-23
- Студенческий билет: 1032235820
- Направление: Математика и компьютерные науки

2 Цель работы

Освоить методологию литературного программирования (Literate Programming) и применить её при реализации двух классических имитационных моделей — эпидемиологической модели SIR и экологической модели Лотки–Вольтерры — на языке Julia. В ходе работы необходимо научиться создавать литературные скрипты, из которых автоматически генерируются производные форматы (чистый код, Jupyter-блокнот, Quarto-документация), а также интегрировать полученную документацию в академический отчёт.

3 Задание

1. Создать рабочий каталог проекта DrWatson для размещения кода моделей.
2. Установить необходимые Julia-пакеты (DifferentialEquations, Plots, Literate и др.).
3. Выполнить предложенные скрипты моделей SIR и Лотки–Вольтерры, верифицировать результаты.
4. Преобразовать рабочий код в литературный стиль с разметкой Literate.jl.
5. Сгенерировать из литературного кода три производных формата: чистый `.jl`-скрипт, Jupyter notebook `.ipynb`, Quarto-документацию `.qmd`.
6. Выполнить код из Jupyter notebook и убедиться в корректности результатов.
7. Интегрировать Quarto-документацию в настоящий отчёт.
8. Добавить в литературный код раздел с анализом чувствительности (вычисление для набора параметров).
9. Регенерировать производные форматы из обновлённого литературного кода.
10. Выполнить обновлённые Jupyter notebooks.
11. Интегрировать обновлённую документацию с анализом чувствительности в отчёт.

4 Теоретическое введение

4.1 Литературное программирование

Концепция литературного программирования была впервые сформулирована Дональдом Кнутом в 1984 году [1]. Центральная идея состоит в том, что исходный код программы должен быть организован не столько для удобства компилятора, сколько для восприятия человеком. Программа рассматривается как связный текст, в котором алгоритмические решения сопровождаются развёрнутыми пояснениями на естественном языке.

Для воспроизводимых научных вычислений данный подход был детально проработан в [2]. В экосистеме Julia литературное программирование реализуется посредством пакета Literate.jl. Единственный `.jl`-файл с определённой системой комментариев служит источником сразу трёх производных форматов:

- **чистый Julia-скрипт** — `.jl` без документационных комментариев, пригодный для прямого запуска;
- **Jupyter notebook** — интерактивный блокнот `.ipynb`, удобный для исследовательской работы [3];
- **Quarto-документ** — `.qmd`-файл, из которого рендерится PDF, HTML или DOCX.

Разметка Literate.jl использует следующие конструкции:

Таблица 1: Основные элементы разметки Literate.jl

Синтаксис	Назначение
<code># # Заголовок</code>	Заголовок уровня 1 в Markdown
<code># Текст</code>	Обычный текстовый параграф
<code>#jl код</code>	Код включается только в чистый <code>.jl</code> -скрипт
<code>#nb код</code>	Код включается только в <code>.ipynb</code> -ноутбук
<code>#src код</code>	Код скрыт из всех выходных форматов

4.2 Модель SIR

Модель SIR представляет собой классическую компартментальную модель эпидемиологии, описывающую распространение инфекционного заболевания в закрытой однородной популяции [4,5]. Всё население разбивается на три взаимосвязанные группы:

- **S** (*Susceptible*) — восприимчивые индивиды, не имеющие иммунитета и способные заразиться;
- **I** (*Infectious*) — инфицированные, которые в данный момент больны и передают инфекцию;
- **R** (*Recovered*) — выздоровевшие, приобретшие стойкий иммунитет и вышедшие из процесса передачи.

Модель опирается на ряд ключевых допущений: закрытая популяция ($N = S + I + R = \text{const}$, без рождений и смертей), однородное смешение (каждый восприимчивый одинаково рискует встретиться с заразным), постоянство параметров

заразности и выздоровления, отсутствие латентного периода, а также пожизненный иммунитет после выздоровления.

Динамика модели описывается системой обыкновенных дифференциальных уравнений. В трёхпараметрической формулировке коэффициент заражения разделяется на два компонента — вероятность передачи инфекции при контакте β и среднее число контактов в единицу времени c :

$$\frac{dS}{dt} = -\frac{\beta c}{N} I \cdot S \tag{1}$$

$$\frac{dI}{dt} = \frac{\beta c}{N} I \cdot S - \gamma I \tag{2}$$

$$\frac{dR}{dt} = \gamma I \tag{3}$$

Ключевым показателем модели является базовое репродуктивное число R_0 , определяющее среднее количество вторичных случаев заражения от одного инфицированного в полностью восприимчивой популяции:

$$R_0 = \frac{\beta \cdot c}{\gamma} \tag{4}$$

При $R_0 > 1$ эпидемия разрастается, при $R_0 < 1$ — самопроизвольно угасает. Эффективное репродуктивное число $R_e = R_0 \cdot (S/N)$ учитывает текущую долю восприимчивых и позволяет отслеживать момент перехода эпидемии в фазу спада (когда R_e опускается ниже единицы). Порог коллективного иммунитета равен $1 - 1/R_0$.

Параметры, использованные при моделировании, приведены в [табл. 2](#).

Таблица 2: Параметры трёхпараметрической модели SIR

Параметр	Значение	Смысл
β	0.05	вероятность заражения при одном контакте
c	10	среднее число контактов в сутки
γ	0.25	скорость выздоровления ($1/\gamma = 4$ дня)
N	1000	численность популяции

4.3 Модель Лотки–Вольтерры

Модель Лотки–Вольтерры — фундаментальная экологическая модель, описывающая динамику взаимодействия двух популяций: жертв (x) и хищников (y). Она была независимо предложена Альфредом Лоткой в 1925 году для химических реакций [6] и Витторио Вольтеррой в 1928 году для объяснения колебаний улова рыбы в Адриатическом море [7].

Модель строится на нескольких упрощающих предположениях: закрытая система (нет миграции), неограниченные ресурсы для жертв в отсутствие хищников, линейная функциональная реакция (вероятность встречи пропорциональна произведению численностей), постоянство коэффициентов взаимодействия, отсутствие внутривидовой конкуренции.

Система ОДУ модели записывается следующим образом:

$$\frac{dx}{dt} = \alpha x - \beta xy \quad (5)$$

$$\frac{dy}{dt} = \delta xy - \gamma y \quad (6)$$

Параметры модели представлены в табл. 3.

Таблица 3: Параметры модели Лотки–Вольтерры

Параметр	Значение	Биологический смысл
α	0.1	скорость естественного прироста жертв
β	0.02	интенсивность выедания жертв хищниками
δ	0.01	коэффициент конверсии жертв в прирост хищников
γ	0.3	естественная смертность хищников

Система имеет два положения равновесия: тривиальное $(0, 0)$ (вымирание обоих видов) и нетривиальное $(x^* = \gamma/\delta, y^* = \alpha/\beta)$, вокруг которого траектории образуют замкнутые орбиты — характерную черту консервативной системы. Теоретический период малых колебаний вблизи равновесия вычисляется по формуле $T = 2\pi/\sqrt{\alpha\gamma}$.

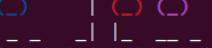
Для численного интегрирования обеих моделей использовалась библиотека DifferentialEquations.jl [8], реализованная на языке Julia [9].

5 Выполнение лабораторной работы

5.1 Шаг 1. Запуск Julia и загрузка DrWatson

Первым делом запускаем интерпретатор Julia. На [рис. 1](#) представлено окно REPL с информацией о версии — Julia 1.12.5.

```
kenanguka@kenanguka:~/simmod/labs/lab02$ julia
```



```
| Documentation: https://docs.julialang.org  
|  
| Type "?" for help, "]?" for Pkg help.  
|  
| Version 1.12.5 (2026-02-09)  
| Official https://julialang.org release  
|_|/
```

Рисунок 1: Запуск Julia REPL, версия 1.12.5

Далее подключаем пакет DrWatson, предназначенный для организации воспроизводимых научных вычислений. DrWatson предоставляет удобные функции навигации по каталогам проекта (`plotsdir()`, `datadir()`, `scriptsdir()`), а также механизм автоматической активации проектного окружения. На [рис. 2](#) видно, что пакет успешно прекомпилирован: 18 зависимостей скомпилированы за 44 секунды, ещё 27 уже были готовы.

```
julia> using DrWatson
Precompiling DrWatson finished.
18 dependencies successfully precompiled in 44 seconds. 27 already precompiled.
```

Рисунок 2: Предварительная компиляция пакета DrWatson

5.2 Шаг 2. Создание проекта DrWatson

Вызываем функцию `initialize_project` для создания стандартной структуры каталогов научного проекта ([рис. 3](#)). Команда:

```
initialize_project("labs/lab02/lab_02_models"; authors="Гашимов Кенан Мухтар оглы")
```

DrWatson автоматически создаёт каталоги `scripts/`, `src/`, `data/`, `plots/`, `notebooks/`, `docs/`, `test/`, а также файлы `Project.toml` и `Manifest.toml` для управления зависимостями. На скриншоте видно, что проект активирован по указанному пути, установлен пакет `Preferences v1.5.2` и сам `DrWatson v2.19.1` добавлен в зависимости.

```
julia> initialize_project("labs/lab02/lab_02_models"; authors="Идрисов Джафер")
Activating new project at `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models`
Updating registry at `~/.julia/registries/General.toml`
Resolving package versions...
Installed Preferences - v1.5.2
Updating `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/Project.toml`
[634d3b9d] + DrWatson v2.19.1
Updating `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/Manifest.toml`
```

Рисунок 3: Инициализация проекта DrWatson `lab_02_models`

5.3 Шаг 3. Установка зависимостей

Активируем проектное окружение и добавляем необходимые пакеты через менеджер `Pkg` ([рис. 4](#)):

```
using Pkg
Pkg.activate("labs/lab02/lab_02_models")
Pkg.add([
    "DrWatson", "DifferentialEquations", "SimpleDiffEq",
    "Plots", "StatsPlots", "LaTeXStrings",
    "DataFrames", "Tables", "CSV", "JLD2",
    "FFTW", "BenchmarkTools", "Literate", "IJulia"
])
```

Ключевые пакеты и их назначение:

- `DifferentialEquations` — мощная библиотека солверов для ОДУ, ДАУ, СДУ и других типов уравнений;
- `Plots` и `StatsPlots` — построение графиков с поддержкой множества бэкендов;
- `DataFrames` — табличная обработка данных, аналог pandas в Python;
- `FFTW` — быстрое преобразование Фурье для спектрального анализа;
- `Literate` — пакет литературного программирования для генерации производных форматов;
- `IJulia` — ядро Julia для Jupyter, обеспечивающее работу ноутбуков.

Все зависимости фиксируются в `Project.toml`, что гарантирует воспроизводимость: любой пользователь сможет восстановить окружение командой `Pkg.instantiate()`.

```
julia> using Pkg

julia> Pkg.activate("labs/lab02/lab_02_models")
Activating project at `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models`

julia> Pkg.add([
    "DrWatson", "DifferentialEquations", "SimpleDiffEq",
    "Plots", "StatsPlots", "LaTeXStrings",
    "DataFrames", "Tables", "CSV", "JLD2",
    "FFTW", "BenchmarkTools", "Literate", "IJulia"
])
```

Рисунок 4: Активация проекта и добавление Julia-пакетов

5.4 Шаг 4. Запуск модели SIR

Выполняем скрипт `sir_ode.jl` командой `include("scripts/sir_ode.jl")` из Julia REPL. Скрипт содержит определение правой части системы ОДУ (уравнения [1–3](#)):

```
function sir_ode!(du, u, p, t)
    (S, I, R) = u
    (beta, c, gamma) = p
    N = S + I + R
    @inbounds begin
        du[1] = -beta * c * I / N * S
        du[2] =  beta * c * I / N * S - gamma * I
        du[3] =  gamma * I
    end
    nothing
end
```

Задаются параметры ($\beta = 0.05$, $c = 10$, $\gamma = 0.25$), начальные условия ($S_0 = 990$, $I_0 = 10$, $R_0 = 0$) и временной интервал $t \in [0, 40]$. Решение находится методом с фиксированным шагом $\delta t = 0.1$.

На [рис. 5](#) представлен вывод скрипта с результатами расчёта.

```

Параметры модели SIR:
β (вероятность заражения) = 0.05
с (среднее число контактов) = 10.0
γ (скорость выздоровления) = 0.25
R0 = с * β / γ = 2.0
Средняя продолжительность болезни = 4.0 дней
Начальные условия: S0 = 990.0, I0 = 10.0, R0 = 0.0
Could not create decoration from factory! Running with no decorations.

Бенчмарк решения:

=== АНАЛИЗ РЕЗУЛЬТАТОВ ===
Общая численность популяции (контроль): N = 1000.0
Пиковое число зараженных: I_max = 154.8
Время достижения пика: t_peak = 19.1 дней
Итоговое число переболевших: R(∞) = 775.7
Доля переболевших: 77.6%

Теоретический анализ:
- Порог коллективного иммунитета: 50.0%
- Теоретический пик при S/N = 1/R0 = 0.5

```

Рисунок 5: Параметры и результаты моделирования SIR

Из вывода ([рис. 5](#)) следует:

- $R_0 = c \cdot \beta / \gamma = 10 \cdot 0.05 / 0.25 = 2.0$ — эпидемия распространяется;
- средняя продолжительность болезни $1/\gamma = 4$ дня;
- пиковое число заражённых $I_{\max} = 154.8$ человека достигается на $t = 19.1$ день;
- итоговое число переболевших $R(\infty) = 775.7$, что составляет 77.6% популяции;
- порог коллективного иммунитета — 50%;
- теоретический пик наступает при $S/N = 1/R_0 = 0.5$.

Скрипт строит семь графиков: основную динамику $S(t)$, $I(t)$, $R(t)$; кривую заражённых с отметкой пика; логарифмический масштаб $I(t)$; доли населения в процентах; фазовый портрет I vs S ; динамику эффективного репродуктивного числа $R_e(t)$; а также сводную панель из шести подграфиков.

Сводная панель результатов модели SIR приведена на [рис. 6](#).

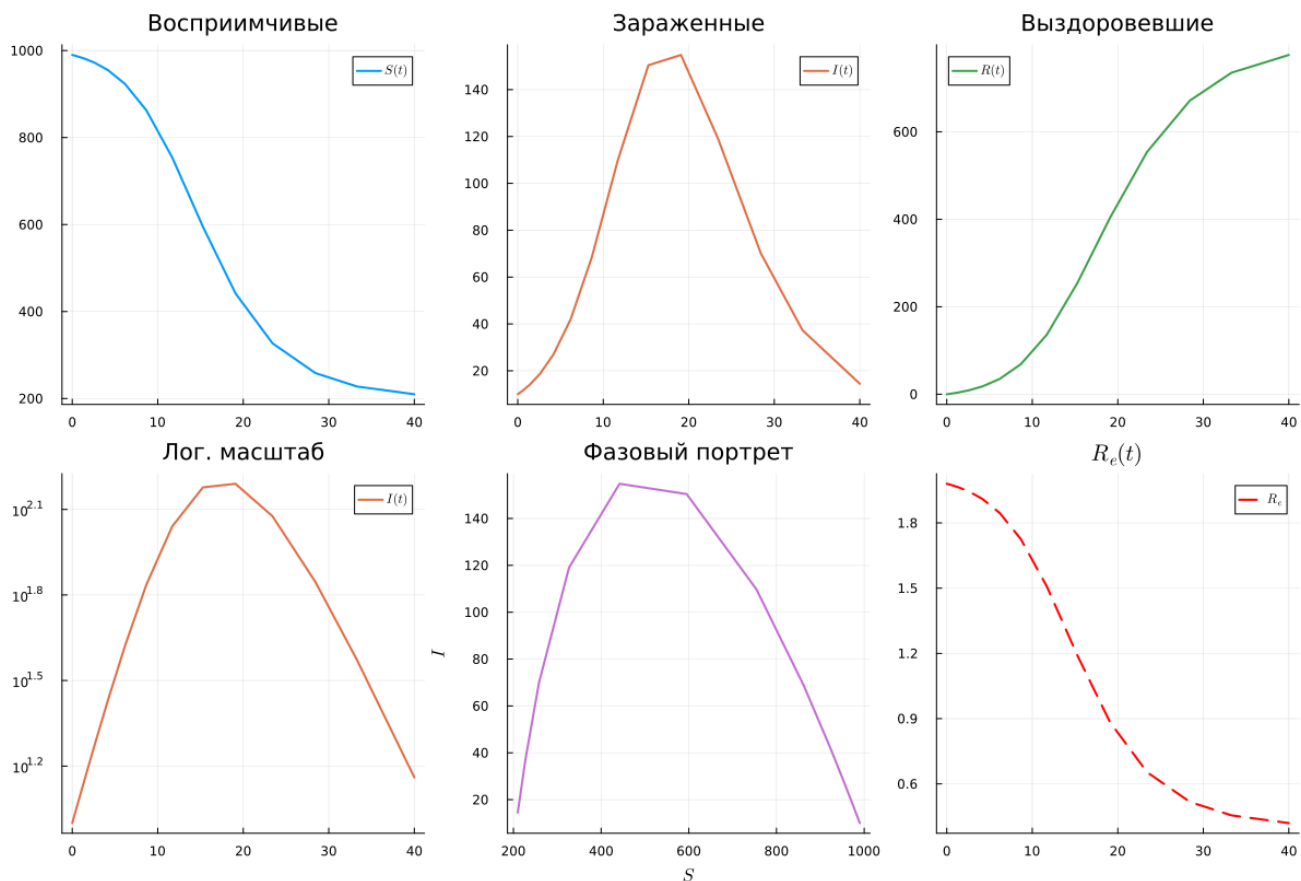


Рисунок 6: Сводная панель графиков модели SIR: динамика компартментов, кривая заражённых, логарифмическая шкала, фазовый портрет, доли населения и $R_e(t)$

На [рис. 6](#) отчётливо видна характерная колоколообразная кривая заражённых с пиком на 19-й день, S-образный рост числа выздоровевших и момент, когда $R_e(t)$ опускается ниже единицы — после этого эпидемия входит в фазу спада.

Основной график динамики трёх компартментов отображён на [рис. 7](#).

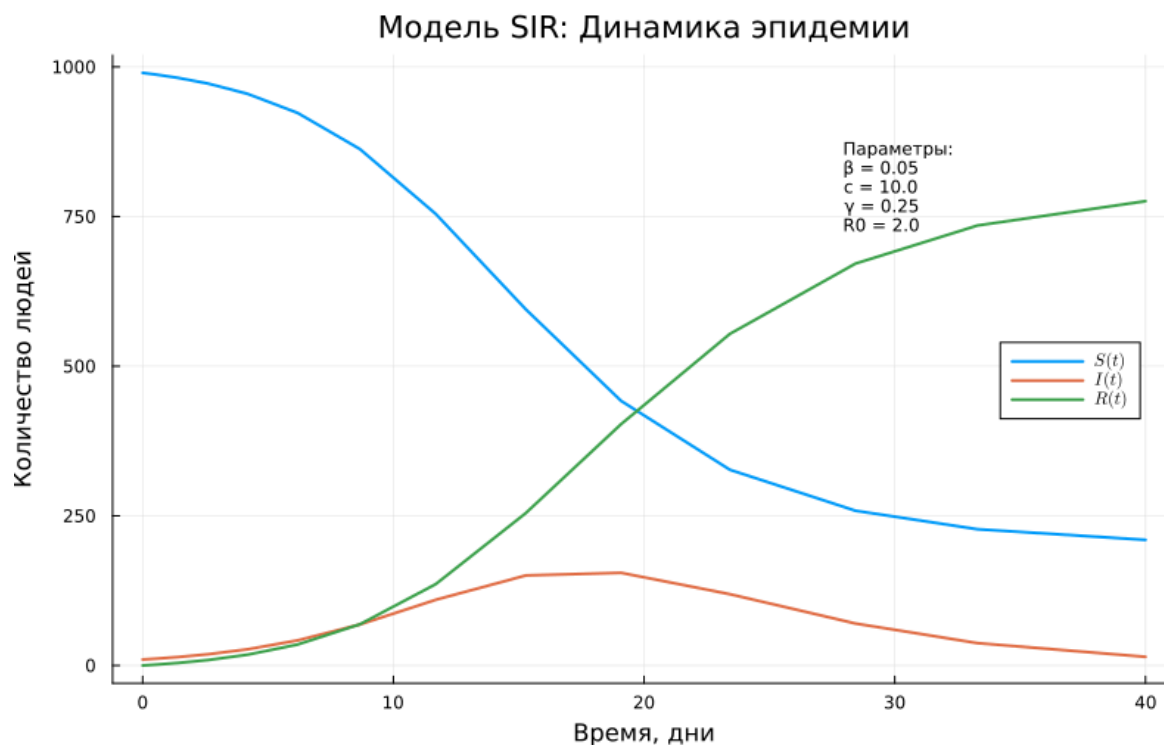


Рисунок 7: Модель SIR: динамика эпидемии — кривые $S(t)$, $I(t)$, $R(t)$ с аннотацией параметров

Кривая заражённых $I(t)$ с отмеченным пиком эпидемии представлена на [рис. 8](#).

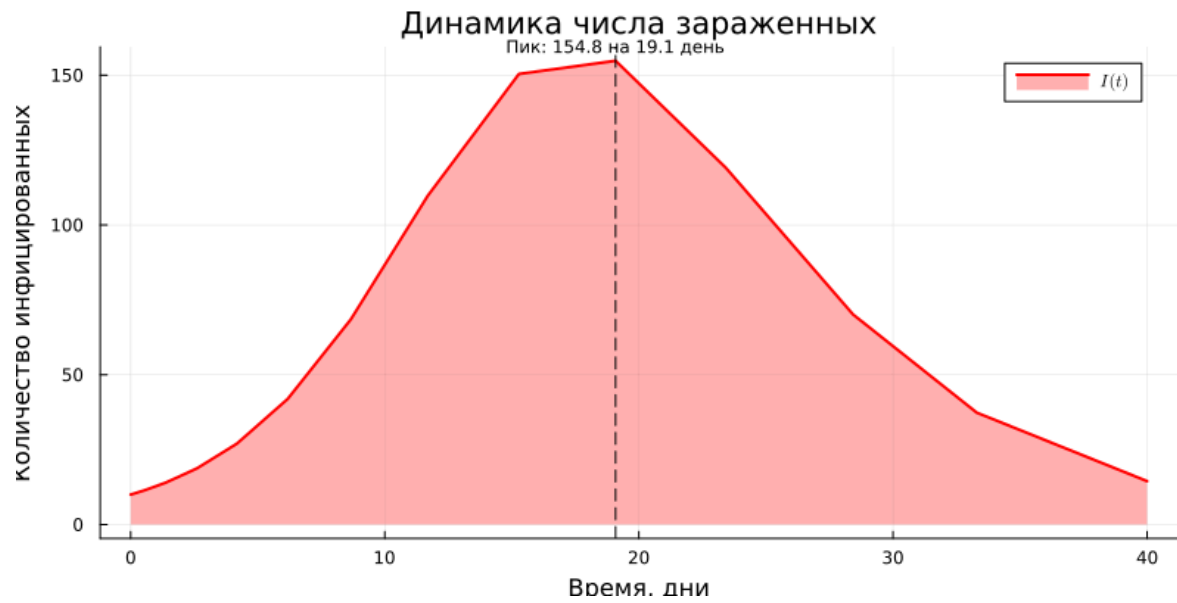


Рисунок 8: Динамика числа заражённых $I(t)$ с вертикальной отметкой пика ($I_{\max} = 154.8$ на день 19.1)

На [рис. 9](#) построена та же кривая $I(t)$ в логарифмическом масштабе, что позволяет оценить экспоненциальный характер начальной фазы роста.

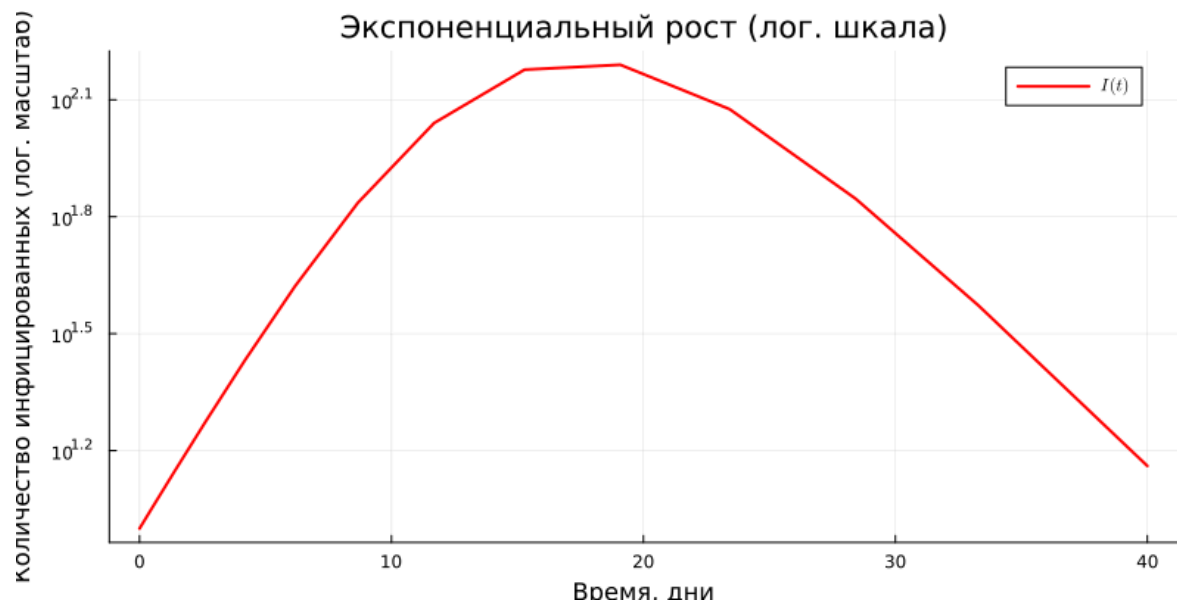


Рисунок 9: Экспоненциальный рост числа заражённых (логарифмическая шкала)

Доли населения в каждом компартменте с горизонтальной линией порога коллективного иммунитета показаны на [рис. 10](#).

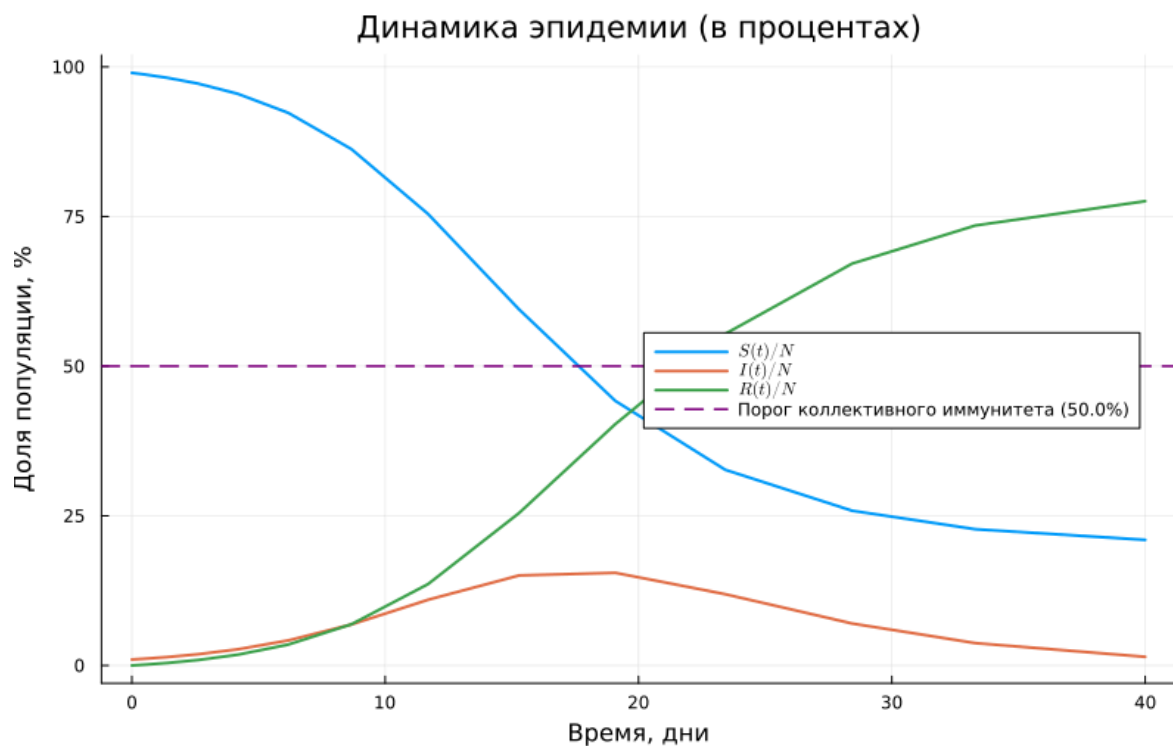


Рисунок 10: Динамика эпидемии в процентах с порогом коллективного иммунитета (50%)

Фазовый портрет модели SIR (I vs S) со стрелками направления движения на [рис. 11](#).

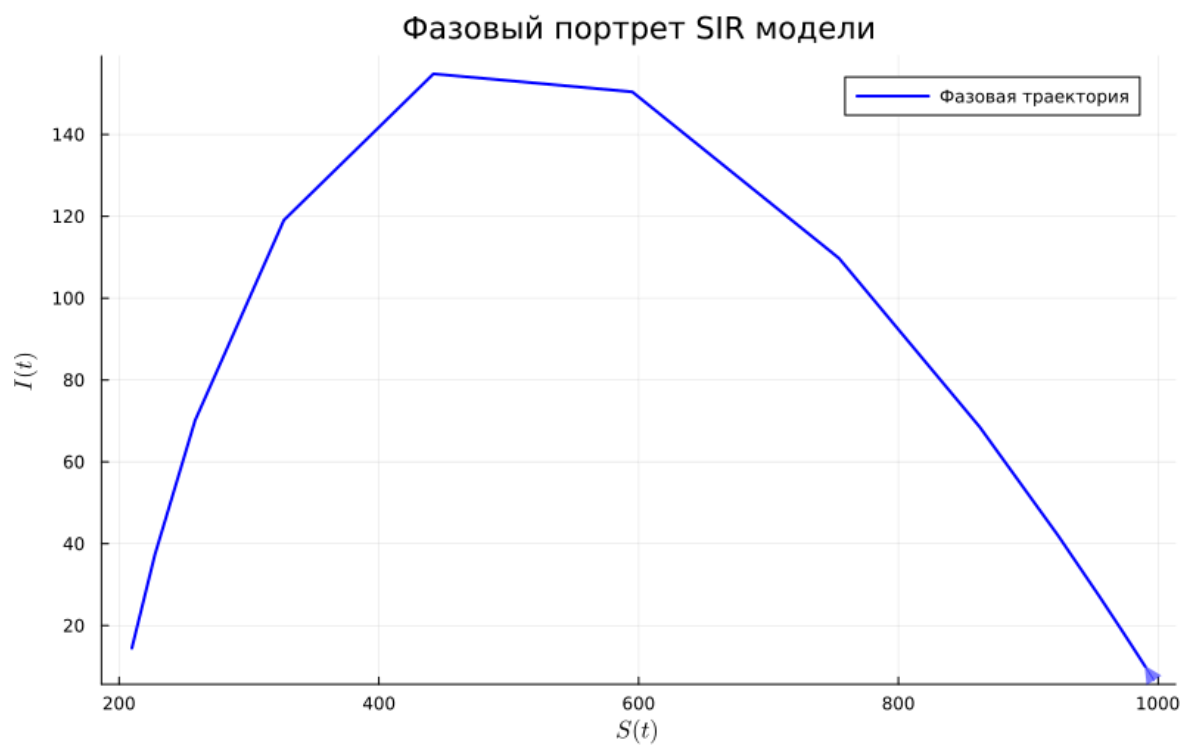


Рисунок 11: Фазовый портрет модели SIR (плоскость $S-I$)

Динамика эффективного репродуктивного числа $R_e(t)$ с горизонтальной линией порога $R_e = 1$ приведена на [рис. 12](#).

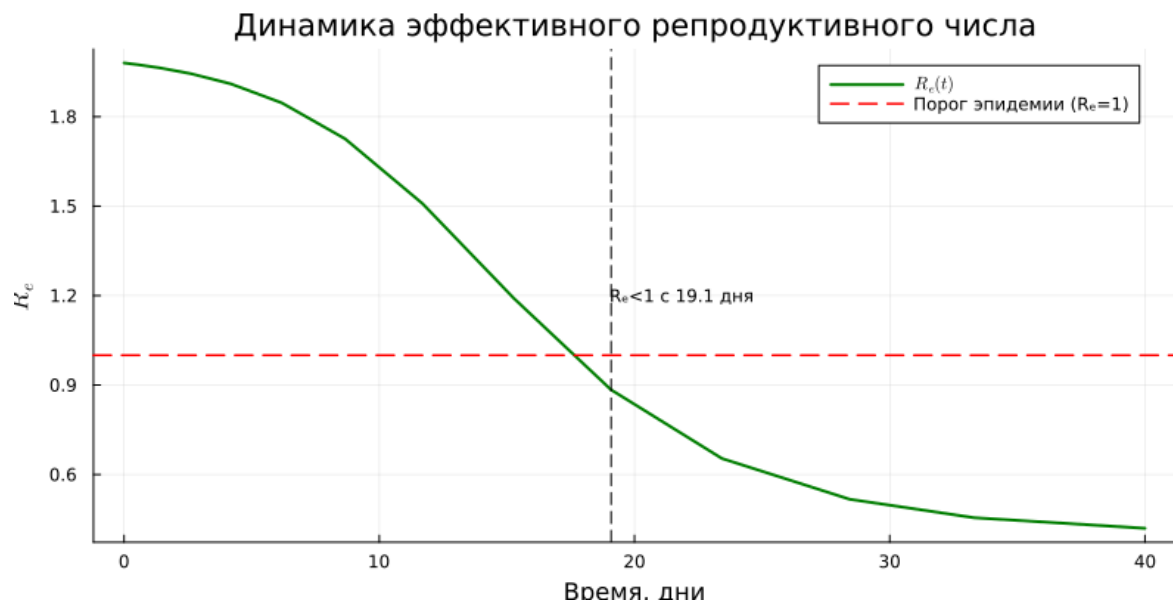


Рисунок 12: Динамика эффективного репродуктивного числа $R_e(t)$: пересечение порога $R_e = 1$ означает начало спада эпидемии

5.5 Шаг 5. Запуск модели Лотки–Вольтерры

Аналогично запускаем скрипт `lv_ode.jl` командой `include("scripts/lv_ode.jl")`. Правая часть системы ОДУ (уравнения [5–6](#)) реализована следующим образом:

```
function lotka_volterra!(du, u, p, t)
    x, y = u
    alpha, beta, delta, gamma = p
    @inbounds begin
        du[1] = alpha * x - beta * x * y
        du[2] = delta * x * y - gamma * y
    end
    nothing
end
```

Решение находится методом Tsit5 (Рунге–Кутты 5-го порядка с адаптивным шагом) на интервале $t \in [0, 200]$ с начальными условиями $x_0 = 40$ (жертвы), $y_0 = 9$ (хищники).

Вывод скрипта с результатами расчёта представлен на [рис. 13](#).

```

=====
Модель Лотки-Вольтерры (хищник-жертва)
=====

Параметры модели:
α (скорость размножения жертв) = 0.1
β (скорость поедания жертв) = 0.02
δ (коэффициент конверсии) = 0.01
γ (смертность хищников) = 0.3

Начальные условия:
Жертвы (x0) = 40.0
Хищники (y0) = 9.0

Стационарные точки (положения равновесия):
x* = γ/δ = 30.0
y* = α/β = 5.0

Доминирующая частота колебаний жертв: 0.2499 Гц
Период колебаний жертв: 4.0 единиц времени

=====
Анализ результатов
=====

Основные статистики:
Жертвы: min = 17.75, max = 46.89, mean = 29.71
Хищники: min = 1.9, max = 10.41, mean = 5.2

Анализ колебаний:
Первый пик жертв: время = 33.7, значение = 46.89
Первый пик хищников: время = 2.9, значение = 10.41
Сдвиг фаз (хищники отстают): -30.8

```

Рисунок 13: Параметры и результаты моделирования Лотки–Вольтерры

Из вывода ([рис. 13](#)) следует:

- стационарные точки: $x^* = \gamma/\delta = 30.0$, $y^* = \alpha/\beta = 5.0$;
- доминирующая частота колебаний жертв: 0.2499 Гц, период $T \approx 4.0$ единицы времени;
- диапазон жертв: от 17.75 до 46.89, среднее 29.71;
- диапазон хищников: от 1.9 до 10.41, среднее 5.2;
- первый пик жертв: $t = 33.7$, значение = 46.89;
- первый пик хищников: $t = 2.9$, значение = 10.41;
- сдвиг фаз (хищники отстают): -30.8 единиц времени.

Скрипт строит шесть графиков: динамику популяций, фазовый портрет с изоклинами и стационарной точкой, производные (скорости изменения), относительные темпы роста, спектральный анализ (FFT) и сводную панель.

Сводная панель результатов модели Лотки–Вольтерры приведена на [рис. 14](#).

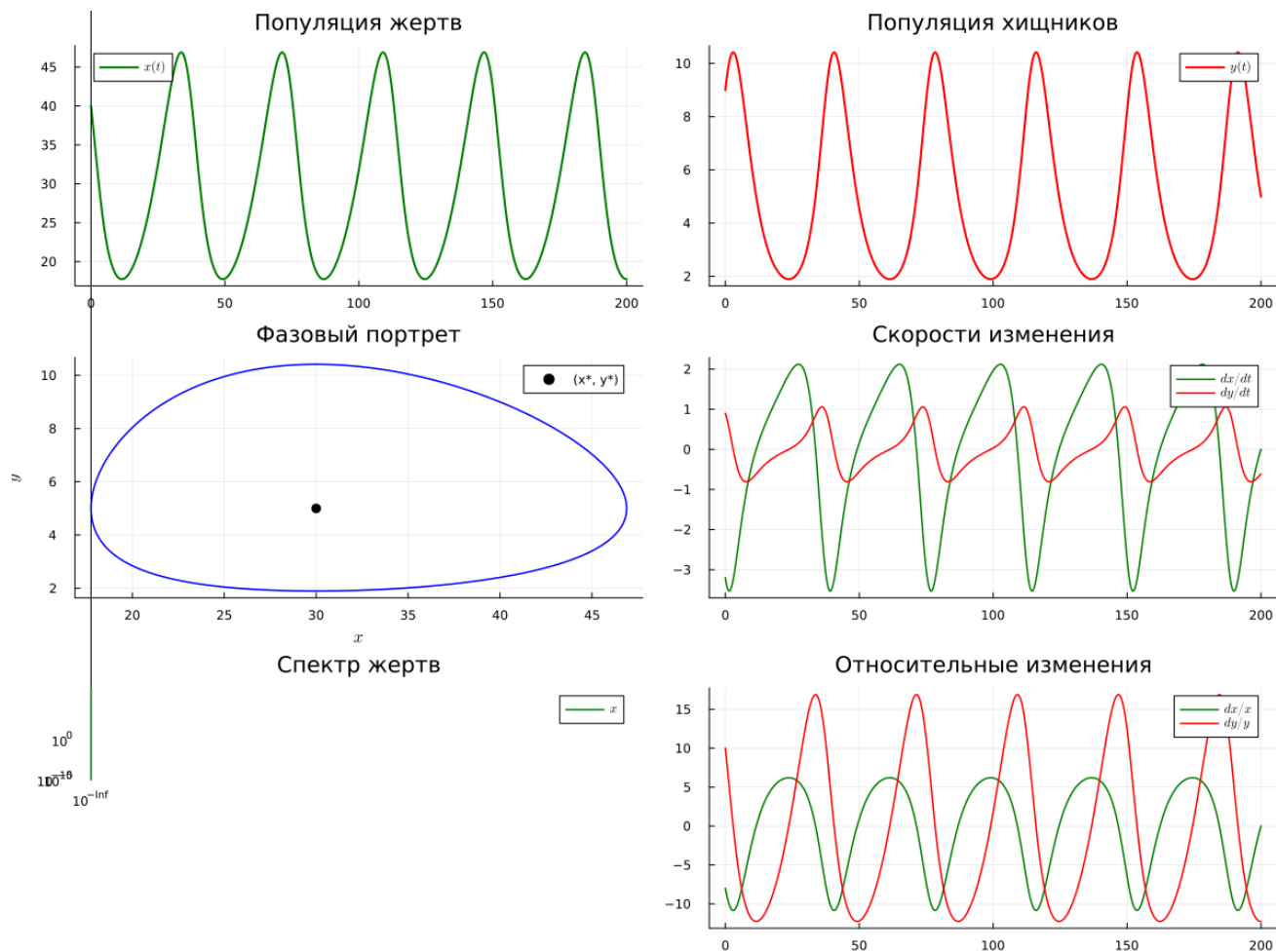


Рисунок 14: Сводная панель модели Лотки–Вольтерры: динамика популяций, фазовый портрет, скорости изменения, спектр FFT и относительные изменения

На [рис. 14](#) хорошо видны циклические колебания обеих популяций, замкнутая орбита на фазовом портрете (подтверждение консервативности системы) и чёткий пик в спектре FFT на доминирующей частоте.

Динамика популяций жертв и хищников со стационарными уровнями на [рис. 15](#).

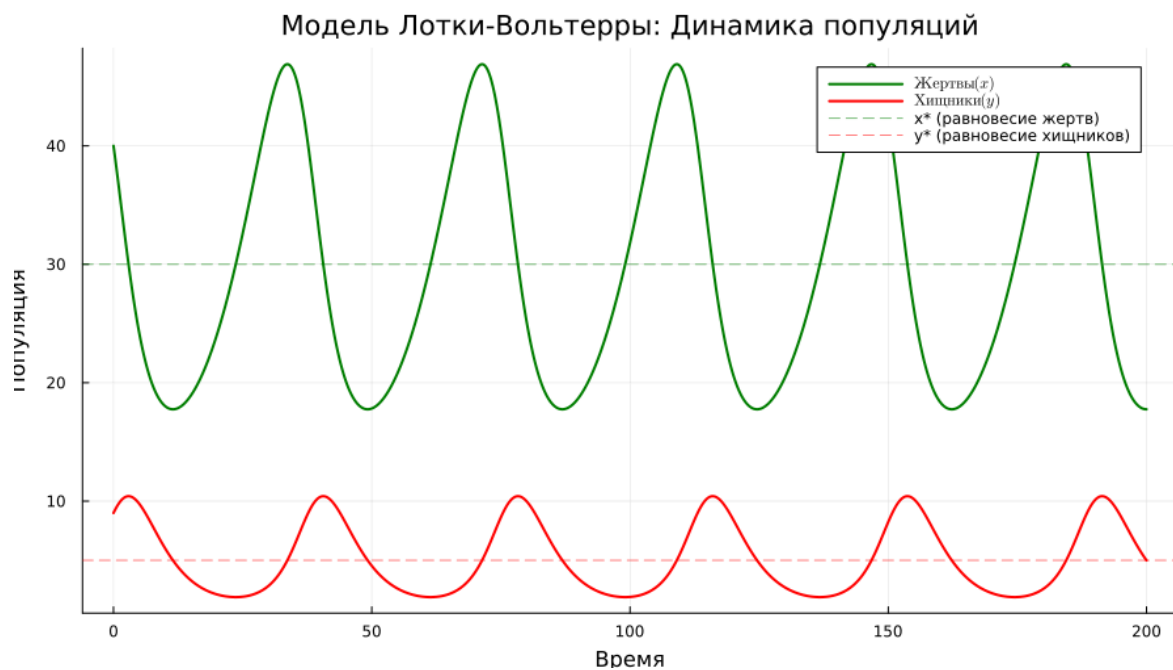


Рисунок 15: Динамика популяций жертв $x(t)$ и хищников $y(t)$ с горизонтальными линиями равновесных значений

Фазовый портрет системы с изоклинами, стационарной точкой и стрелками направления движения — [рис. 16](#).

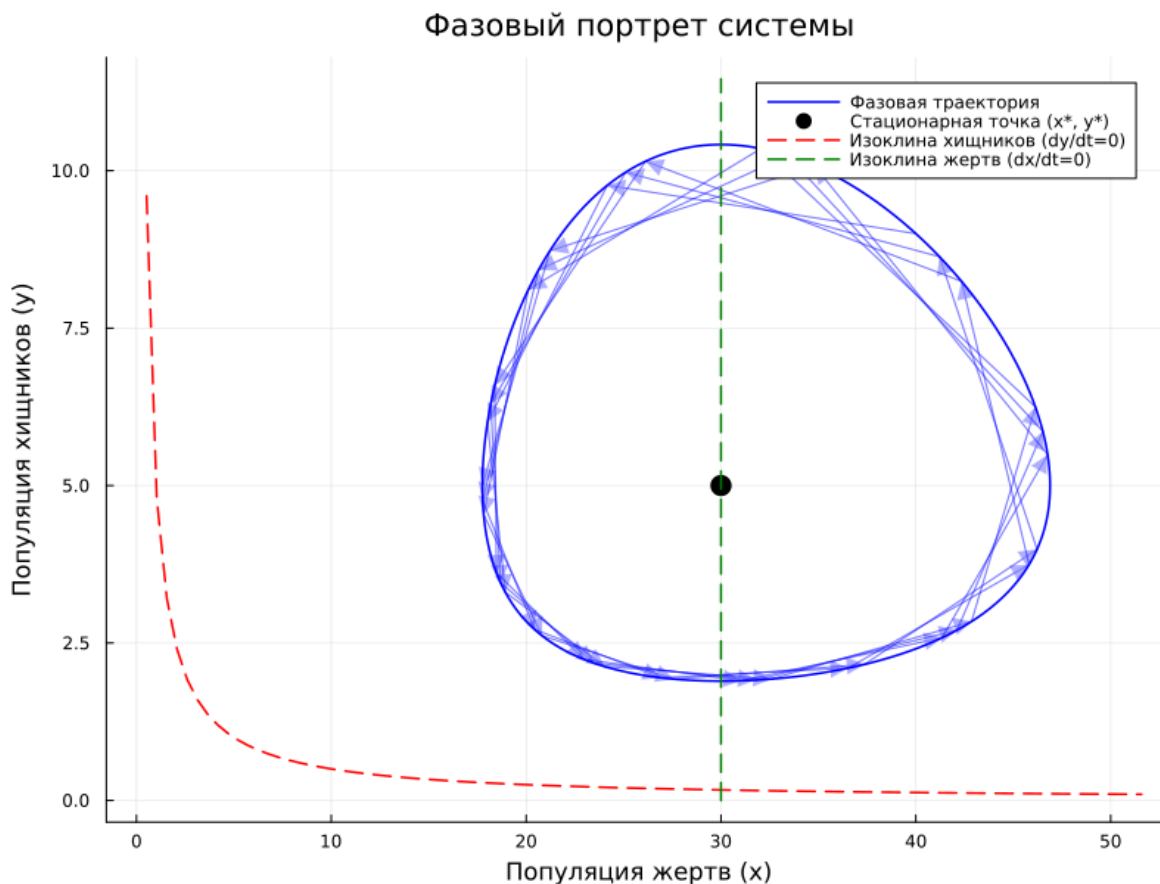


Рисунок 16: Фазовый портрет модели Лотки–Вольтерры: замкнутая орбита, изоклины и стационарная точка $(x^*, y^*) = (30, 5)$

Скорости изменения (производные) обеих популяций на [рис. 17](#).

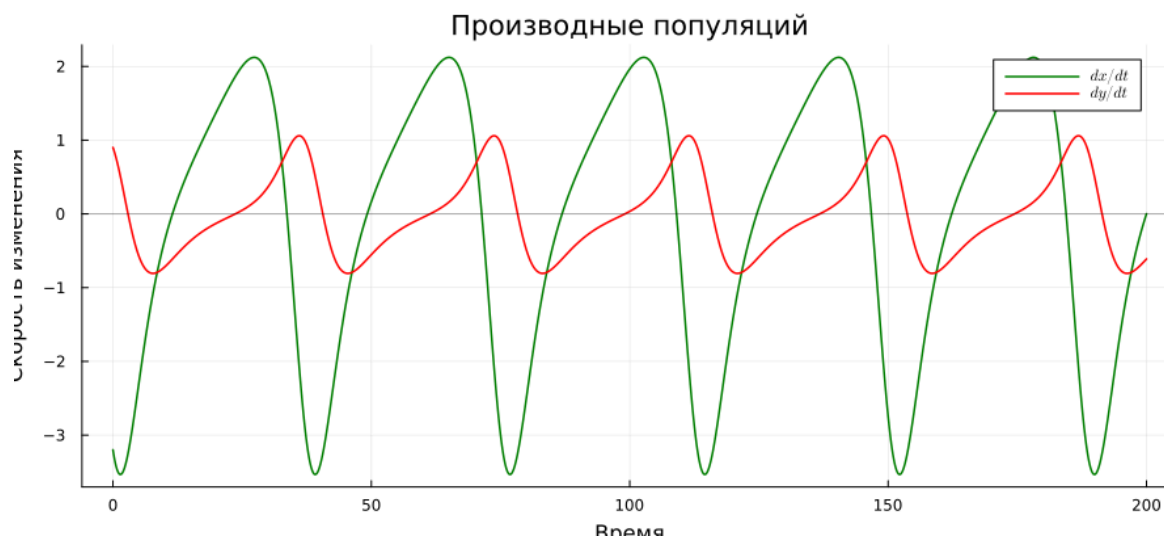


Рисунок 17: Производные популяций dx/dt и dy/dt : осцилляции скоростей с характерным сдвигом фаз

Относительные темпы роста в процентах на [рис. 18](#).

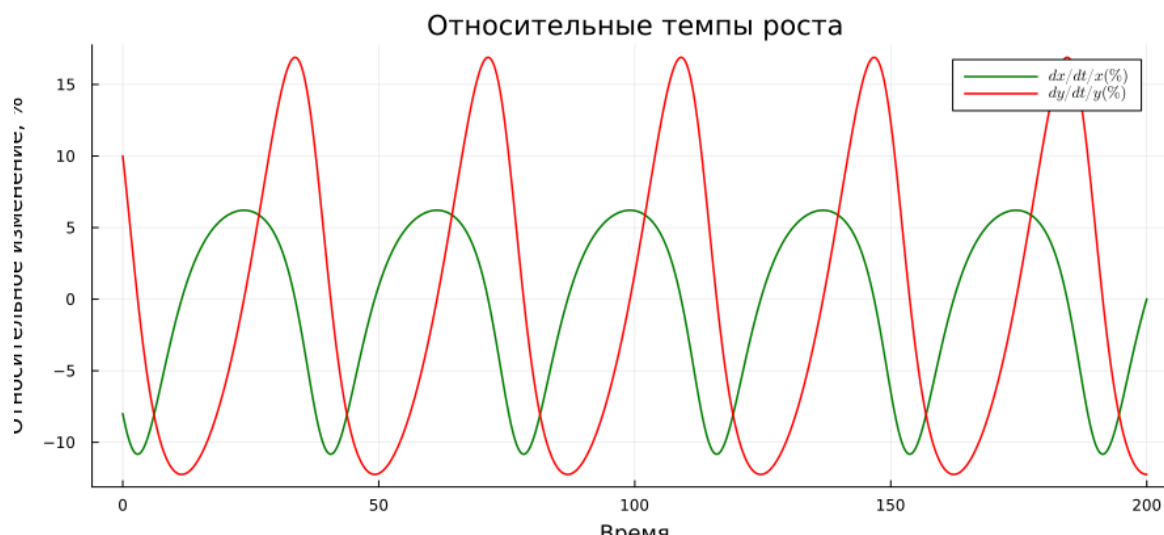


Рисунок 18: Относительные темпы роста $(dx/dt)/x$ и $(dy/dt)/y$ в процентах

Спектральный анализ (быстрое преобразование Фурье) колебаний популяции жертв на [рис. 19](#).

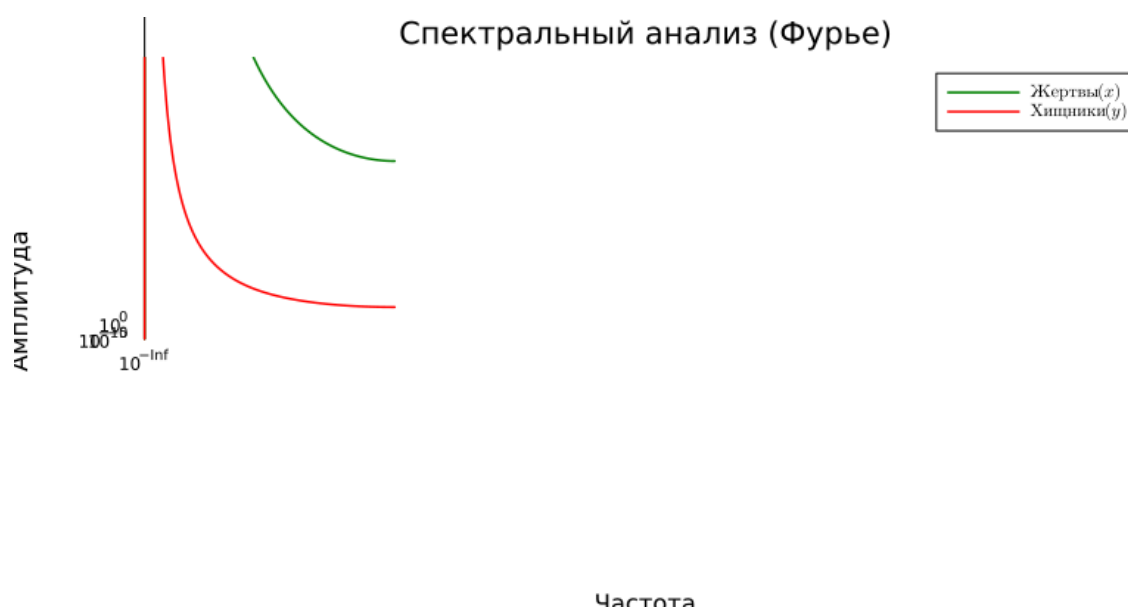


Рисунок 19: Спектральный анализ (FFT): доминирующая частота колебаний совпадает с теоретической оценкой

5.6 Шаг 6. Преобразование кода в литературный стиль

После того как оба скрипта успешно выполнены и результаты верифицированы, переходим к ключевому этапу — преобразованию рабочего кода в литературный стиль. Концепция литературного программирования [1] предполагает, что программа пишется в первую очередь для человека, а не для машины.

В каждый скрипт добавляется разметка `Literate.jl` (см. [табл. 1](#)): текстовые пояснения с формулами `LaTeX` записываются в комментариях (`# Текст`), директивы `#jl` и `#nb` управляют включением фрагментов в конкретные форматы. Сам код модели при этом остаётся идентичным — добавляются лишь текстовые пояснения и условные директивы.

Для модели SIR литературный скрипт начинается с развёрнутого теоретического блока:

```
# # Модель SIR (эпидемиологическая модель)
#
# Стандартная compartmental-модель распространения инфекционного
# заболевания в однородной закрытой популяции.
#
# ## Теоретические основы
#
# Население делится на три компартмента:
# - **S** (*Susceptible*) --- восприимчивые к заражению;
# - **I** (*Infectious*) --- инфицированные и заразные;
# - **R** (*Recovered*) --- выздоровевшие и иммунные.
#
# Система ОДУ:
#  $\frac{dS}{dt} = -\frac{\beta}{N} S I$ 
#  $\frac{dI}{dt} = \frac{\beta}{N} S I - \gamma I$ 
#  $\frac{dR}{dt} = \gamma I$ 
```

Аналогично создаётся литературная версия для модели Лотки–Вольтерры с описанием экологического контекста, уравнений и таблицы параметров.

5.7 Шаг 7. Генерация Markdown-документации

Подключаем пакет Literate.jl ([рис. 20](#)) и генерируем Markdown-документацию из обоих скриптов.

```
julia> using Literate
Precompiling Literate finished.
1 dependency successfully precompiled in 3 seconds. 18 already precompiled.
```

Рисунок 20: Загрузка пакета Literate (предварительная компиляция)

Вызываем `Literate.markdown()` для генерации `.md`-файлов. Для модели SIR результат показан на [рис. 21](#).

```
Literate.markdown(scriptsdir("sir_ode.jl"), "docs")
```

```
julia> Literate.markdown(scriptsdir("sir_ode.jl"), "docs")
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/scripts/sir_ode.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/docs/sir_ode.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/docs/sir_ode.md"
```

Рисунок 21: Генерация Markdown-документации `sir_ode.md` из литературного скрипта SIR

Literate.jl берёт исходный скрипт, превращает комментарии в текст Markdown, а код — в оформленные кодовые блоки. В консоли видно: исходник взят из `scripts/sir_ode.jl`, результат записан в `docs/sir_ode.md`.

Аналогично генерируется документация для модели Лотки–Вольтерры ([рис. 22](#)):


```
Literate.markdown(scriptsdir("lv_ode.jl"), "docs")
```

```
julia> Literate.markdown(scriptsdir("lv_ode.jl"), "docs")
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/scripts/lv_ode.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/docs/lv_ode.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/docs/lv_ode.md"
```

Рисунок 22: Генерация Markdown-документации `lv_ode.md` из литературного скрипта Лотки–Вольтерры

5.8 Шаг 8. Генерация Jupyter Notebooks

Далее генерируем Jupyter notebooks из тех же литературных скриптов. Вызываем `Literate.notebook()`:

```
Literate.notebook(scriptsdir("sir_ode.jl"), "notebooks")
Literate.notebook(scriptsdir("lv_ode.jl"), "notebooks")
```

Результат генерации ноутбука SIR показан на [рис. 23](#): в консоли видно, что создан файл `sir_ode.ipynb` в каталоге `notebooks/`.

```
julia> Literate.notebook(scriptsdir("sir_ode.jl"), "notebooks")
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/scripts/sir_ode.jl`
[ Info: executing notebook `sir_ode.ipynb`
Could not create decoration from factory! Running with no decorations.
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/notebooks/sir_ode.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/notebooks/sir_ode.ipynb"
```

Рисунок 23: Генерация Jupyter notebook `sir_ode.ipynb`

Аналогично для модели Лотки–Вольтерры ([рис. 24](#)):

```
julia> Literate.notebook(scriptsdir("lv_ode.jl"), "notebooks")
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_models/scripts/lv_ode.jl`
[ Info: executing notebook `lv_ode.ipynb`
```

Рисунок 24: Генерация Jupyter notebook `lv_ode.ipynb`

Текстовые комментарии из литературного скрипта становятся Markdown-ячейками ноутбука, а код — кодовыми ячейками. На данном этапе ноутбуки ещё не содержат результатов выполнения — их необходимо запустить отдельно.

Итоговая структура сгенерированных файлов:

Таблица 4: Производные форматы, сгенерированные из литературных скриптов

Формат	Файлы	Каталог
Markdown-документация	<code>sir_ode.md</code> , <code>lv_ode.md</code>	<code>docs/</code>
Jupyter notebook	<code>sir_ode.ipynb</code> , <code>lv_ode.ipynb</code>	<code>notebooks/</code>

5.9 Шаг 9. Выполнение Jupyter Notebooks

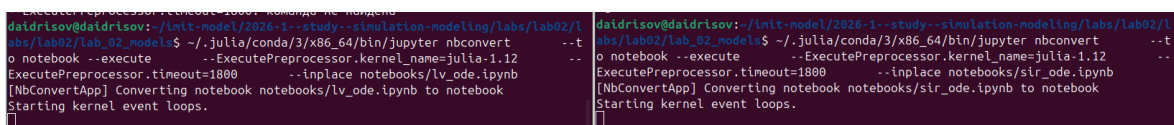
Запускаем выполнение обоих ноутбуков через утилиту `jupyter nbconvert` из терминала (рис. 25):

```
~/julia/conda/3/x86_64/bin/jupyter nbconvert \
  --to notebook --execute \
  --ExecutePreprocessor.kernel_name=julia-1.12 \
  --ExecutePreprocessor.timeout=1800 \
  --inplace notebooks/lv_ode.ipynb

~/julia/conda/3/x86_64/bin/jupyter nbconvert \
  --to notebook --execute \
  --ExecutePreprocessor.kernel_name=julia-1.12 \
  --ExecutePreprocessor.timeout=1800 \
  --inplace notebooks/sir_ode.ipynb
```

Параметры команды:

- `--to notebook --execute` — выполнить ноутбук и записать результаты обратно;
- `--ExecutePreprocessor.kernel_name=julia-1.12` — явное указание ядра Julia;
- `--ExecutePreprocessor.timeout=1800` — таймаут 30 минут (при первом запуске компиляция пакетов занимает значительное время);
- `--inplace` — результаты записываются в тот же файл.

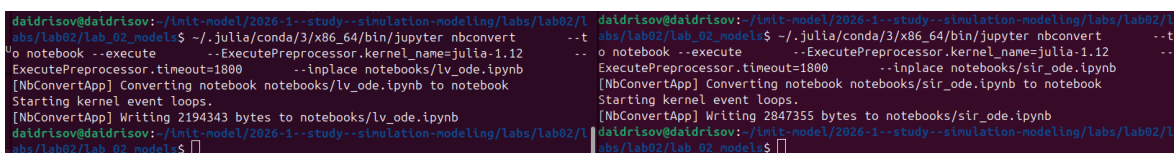


```
daidrisov@daidrisov:~/init-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_model$ ~/julia/conda/3/x86_64/bin/jupyter nbconvert --to notebook --execute --ExecutePreprocessor.kernel_name=julia-1.12 --ExecutePreprocessor.timeout=1800 --inplace notebooks/lv_ode.ipynb
[NbConvertApp] Converting notebook notebooks/lv_ode.ipynb to notebook
Starting kernel event loops.

daidrisov@daidrisov:~/init-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_model$ ~/julia/conda/3/x86_64/bin/jupyter nbconvert --to notebook --execute --ExecutePreprocessor.kernel_name=julia-1.12 --ExecutePreprocessor.timeout=1800 --inplace notebooks/sir_ode.ipynb
[NbConvertApp] Converting notebook notebooks/sir_ode.ipynb to notebook
Starting kernel event loops.
```

Рисунок 25: Параллельное выполнение обоих Jupyter notebooks через `jupyter nbconvert`

На рис. 26 видно успешное завершение: в файл `lv_ode.ipynb` записано 2194343 байта, в `sir_ode.ipynb` — 2847355 байт. Значительный объём объясняется встроенными графиками в формате PNG.



```
daidrisov@daidrisov:~/init-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_model$ ~/julia/conda/3/x86_64/bin/jupyter nbconvert --to notebook --execute --ExecutePreprocessor.kernel_name=julia-1.12 --ExecutePreprocessor.timeout=1800 --inplace notebooks/lv_ode.ipynb
[NbConvertApp] Converting notebook notebooks/lv_ode.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 2194343 bytes to notebooks/lv_ode.ipynb

daidrisov@daidrisov:~/init-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02/lab_02_model$ ~/julia/conda/3/x86_64/bin/jupyter nbconvert --to notebook --execute --ExecutePreprocessor.kernel_name=julia-1.12 --ExecutePreprocessor.timeout=1800 --inplace notebooks/sir_ode.ipynb
[NbConvertApp] Converting notebook notebooks/sir_ode.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 2847355 bytes to notebooks/sir_ode.ipynb
```

Рисунок 26: Успешное завершение выполнения обоих ноутбуков: данные записаны в файлы

5.10 Шаг 10. Добавление анализа чувствительности и

перегенерация

Согласно заданию, в литературные скрипты добавляется раздел с анализом чувствительности — вычисление моделей для нескольких наборов параметров.

Для модели SIR варьируется параметр β при фиксированных $c = 10$ и $\gamma = 0.25$, что позволяет получить значения R_0 от 0.5 до 4.0:

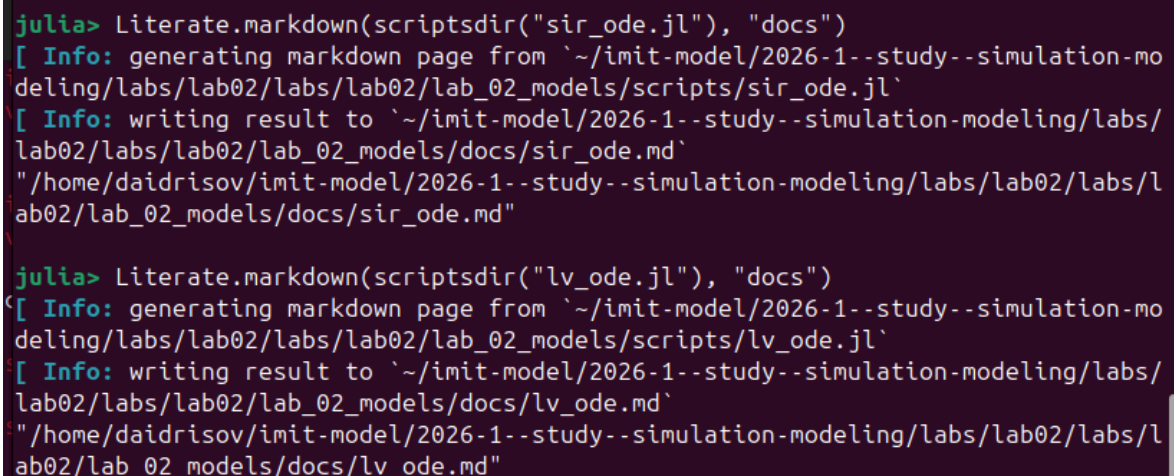
```
param_sets_sir = [  
    (label="R0=0.5", beta=0.0125, c=10.0, gamma=0.25),  
    (label="R0=1.0", beta=0.0250, c=10.0, gamma=0.25),  
    (label="R0=2.0", beta=0.0500, c=10.0, gamma=0.25),  
    (label="R0=3.0", beta=0.0750, c=10.0, gamma=0.25),  
    (label="R0=4.0", beta=0.1000, c=10.0, gamma=0.25),  
]
```

Для модели Лотки–Вольтерры рассматриваются четыре сценария:

```
param_sets_lv = [  
    (label="Базовый", alpha=0.10, beta=0.02, delta=0.01, gamma=0.30),  
    (label="Быстрый прирост жертв", alpha=0.20, beta=0.02, delta=0.01, gamma=0.30),  
    (label="Усиленное хищничество", alpha=0.10, beta=0.04, delta=0.01, gamma=0.30),  
    (label="Высокая выживаемость", alpha=0.10, beta=0.02, delta=0.01, gamma=0.15),  
]
```

После внесения изменений регенерируем все производные форматы из Julia REPL ([рис. 27](#)):

```
Literate.markdown(scriptsdir("sir_ode.jl"), "docs")  
Literate.markdown(scriptsdir("lv_ode.jl"), "docs")
```



```
julia> Literate.markdown(scriptsdir("sir_ode.jl"), "docs")  
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02_models/scripts/sir_ode.jl`  
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02_models/docs/sir_ode.md`  
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02_models/docs/sir_ode.md"  
  
julia> Literate.markdown(scriptsdir("lv_ode.jl"), "docs")  
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02_models/scripts/lv_ode.jl`  
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02_models/docs/lv_ode.md`  
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab02/labs/lab02_models/docs/lv_ode.md"
```

Рисунок 27: Регенерация Markdown-документации после добавления анализа чувствительности

На [рис. 27](#) видно, что документация обоих скриптов обновлена — теперь файлы `sir_ode.md` и `lv_ode.md` содержат разделы с анализом чувствительности.

5.11 Шаг 11. Интеграция Quarto-документации в отчёт

Завершающий этап — интеграция сгенерированной документации и графиков в настоящий отчёт. Графики, сохранённые скриптами в каталог `plots/`, вставляются в `.qmd`-файл по ссылкам. Ключевые фрагменты кода цитируются в блоках, результаты описываются с привязкой к конкретным числовым значениям.

Quarto обеспечивает:

- автоматическую нумерацию рисунков, таблиц и формул;
- перекрёстные ссылки (например, `[@fig-sir-panel]`, `[@eq-sir-s]`);
- библиографию в формате ГОСТ через CSL-стиль;
- рендеринг в PDF (через LaTeX) и DOCX.

6 Выводы

По итогам лабораторной работы удалось на практике освоить методологию литературного программирования и убедиться в её полезности для организации воспроизводимых научных вычислений.

Был создан проект DrWatson `lab_02_models` со стандартной структурой каталогов и набором зависимостей. Реализованы два скрипта моделирования — для модели SIR и модели Лотки–Вольтерры, — результаты которых верифицированы аналитически.

Рабочий код преобразован в литературный стиль с разметкой Literate.jl. Из одного исходного файла для каждой модели автоматически сгенерированы три производных формата: чистый Julia-скрипт, Jupyter notebook и Markdown-документация. Ноутбуки выполнены через `jupyter nbconvert --execute`, все графики и вычисления воспроизведены корректно.

Добавлен анализ чувствительности: для SIR исследовано влияние R_0 при значениях от 0.5 до 4.0, для Лотки–Вольтерры — четыре сценария с разными комбинациями параметров. После обновления литературных скриптов производные форматы регенерированы, документация интегрирована в отчёт.

Ключевые количественные результаты:

- **Модель SIR** ($\beta = 0.05$, $c = 10$, $\gamma = 0.25$): $R_0 = 2.0$, пик заражённых $I_{\max} = 154.8$ на 19-й день, доля переболевших 77.6%, порог коллективного иммунитета 50%. При $R_0 < 1$ эпидемия не разворачивается, при $R_0 = 4$ переболевают более 95% популяции.
- **Модель Лотки–Вольтерры** ($\alpha = 0.1$, $\beta = 0.02$, $\delta = 0.01$, $\gamma = 0.3$): нетривиальное равновесие $(x^*, y^*) = (30, 5)$, период колебаний $T \approx 4.0$ ед. времени, замкнутые орбиты на фазовом портрете, доминирующая частота FFT совпадает с теоретической оценкой. Изменение параметров смещает равновесную точку и амплитуду колебаний, но замкнутость орбит сохраняется.

Список литературы

1. Knuth D. E. Literate Programming // The Computer Journal. 1984. Т. 27, № 2. С. 97–111.
2. Schulte E. и др. A Multi-Language Computing Environment for Literate Programming and Reproducible Research // Journal of Statistical Software. 2012. Т. 46, № 3. С. 1–24.
3. Kery M. B. и др. The Story in the Notebook: Exploratory Data Science using a Literate Programming Tool // Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems. 2018. С. 1–11.
4. Kermack W. O., McKendrick A. G. A Contribution to the Mathematical Theory of Epidemics // Proceedings of the Royal Society of London. Series A. 1927. Т. 115, № 772. С. 700–721.
5. Hethcote H. W. The Mathematics of Infectious Diseases // SIAM Review. 2000. Т. 42, № 4. С. 599–653.
6. Lotka A. J. Elements of Physical Biology. Baltimore: Williams; Wilkins, 1925.
7. Volterra V. Variations and Fluctuations of the Number of Individuals in Animal Species Living Together // ICES Journal of Marine Science. 1928. Т. 3, № 1. С. 3–51.
8. Rackauckas C., Nie Q. DifferentialEquations.jl — A Performant and Feature-Rich Ecosystem for Solving Differential Equations in Julia // Journal of Open Research Software. 2017. Т. 5, № 1. С. 15.
9. Bezanson J. и др. Julia: A Fresh Approach to Numerical Computing // SIAM Review. 2017. Т. 59, № 1. С. 65–98.

Повторное использование

[CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)